

7.5

Числа

00:00–00:18

Привет! В этом видео мы поговорим с вами о числах — как целых, так и числах с плавающей точкой, а также обсудим, для чего существует так много разновидностей примитивов.

00:18–02:03

И начнём с целых чисел.

Самое маленькое целое число — это **байт**. Байт занимает в памяти 8 бит и может принимать всего 256 значений — от -128 до 127 , потому что в байте возможно всего 256 комбинаций битов, то есть нулей и единиц.

Следующее целое число — число **short**, что в переводе с английского означает «короткий». Оно занимает в памяти 16 бит. 16 бит — это 16 нулей или единиц, то есть количество комбинаций равно 2^{16} , или 256^2 , то есть 65 536 значений. Если эти значения распределить относительно нуля, то получится диапазон от $-32\,768$ до $32\,767$.

С целым числом **int** вы уже хорошо знакомы: оно занимает в памяти 32 бита, или 4 байта, и, соответственно, может принимать значения от -2 миллиардов с небольшим до $+2$ миллиардов (тоже с небольшим).

Откуда взялись эти цифры? 32 бита — это 2^{32} комбинаций, это примерно 4 миллиарда комбинаций, и если разделить эту цифру пополам, то как раз получится какой диапазон.

Самое большое целое число — это число типа long, что в переводе с английского значит «длинный». Оно занимает в памяти 64 бита, или 8 байт, и может принимать значение примерно от $-9,2 * 10^{18}$ до примерно $+9,2 * 10^{18}$. 10^{18} — это миллиард миллиардов, то есть эти числа могут состоять максимум из 19 цифр.

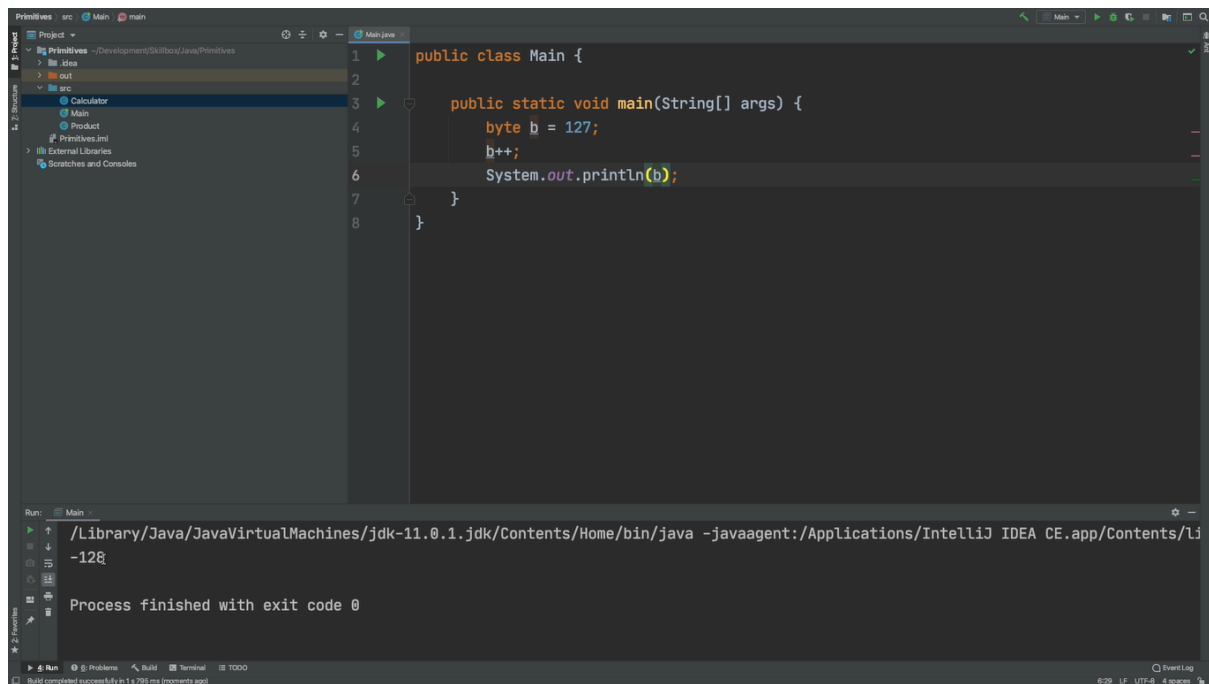
02:03–04:42

Переполнение

Важно не забывать, что при превышении числа максимально допустимого значения может произойти так называемое **переполнение** — это ситуация, при которой к максимальному значению числа прибавляется ещё какое-то число и значение числа становится совсем другим.

Например, если мы возьмём переменную типа байта, максимальное значение не будет больше 127. Давайте посмотрим, что произойдёт, если это значение увеличить.

Пишем: `byte b = 127`, далее пишем: `b++`, то есть увеличиваем значение переменной на единицу, и выводим результат в консоль. Переменная теперь равна -128 , то есть произошло переполнение.



Таким образом, по сути, если мы увеличим максимальное число на единицу, то оно становится минимальным, и дальше мы сможем снова увеличивать его до тех пор, пока оно не станет максимальным. Так происходит со всеми целыми числами.

Переполнение

byte = 8 bit



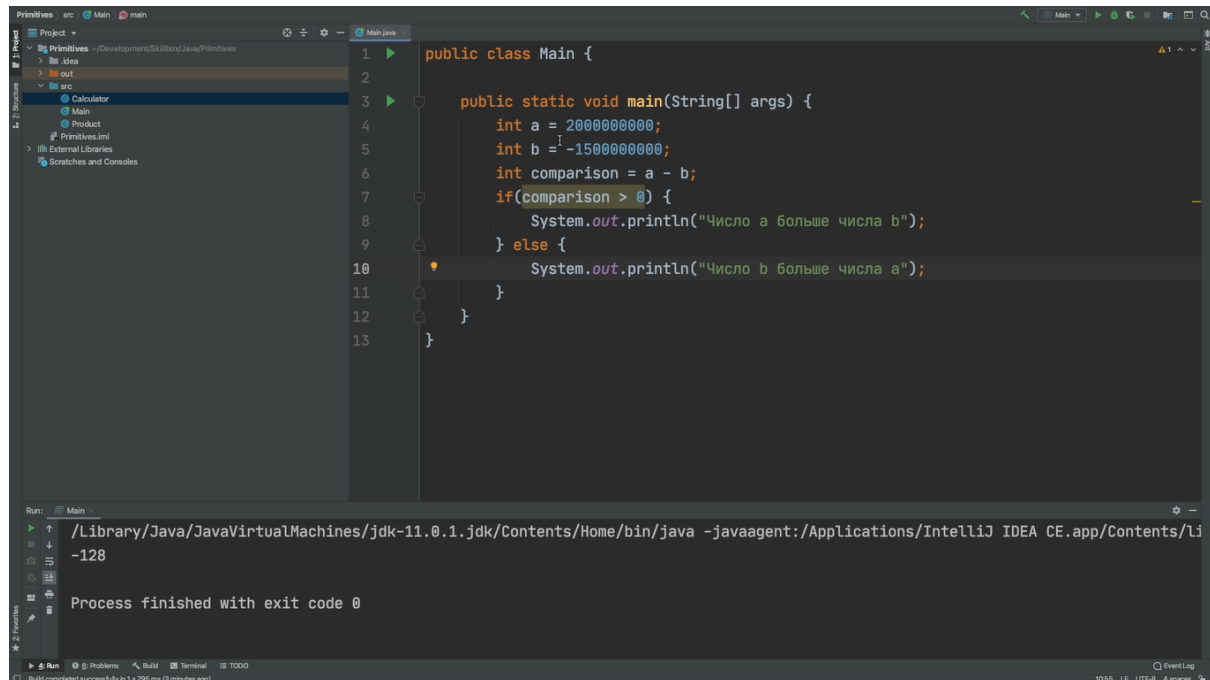
```
byte b = 127;
```



Важно помнить о переполнении, когда вы производите любые вычисления.

Например, довольно частая ошибка — сравнение двух целых чисел методом вычитания. Допустим, есть два целых числа — число a , равное 2 млрд, и число b , которое равно $-1,5$ млрд, и вы вычисляете разницу между ними путём вычитания.

Создаёте ещё одну переменную `int comparison = a - b;` и дальше пишете такой код.

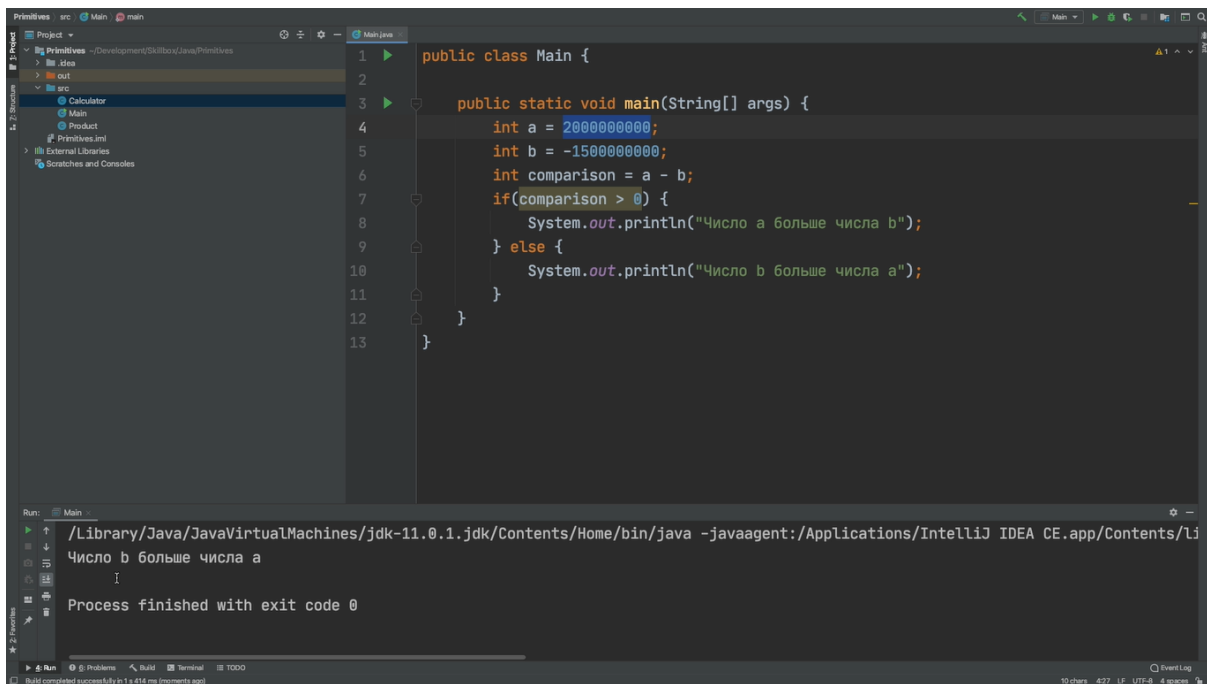


The screenshot shows an IDE window with a project named 'Primitives'. The 'src' folder contains 'Main.java'. The code in 'Main.java' is as follows:

```
1 public class Main {
2
3     public static void main(String[] args) {
4         int a = 2000000000;
5         int b = -1500000000;
6         int comparison = a - b;
7         if (comparison > 0) {
8             System.out.println("Число a больше числа b");
9         } else {
10            System.out.println("Число b больше числа a");
11        }
12    }
13 }
```

The 'Run' window at the bottom shows the command executed: `/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/lib`. The output is `-128`. Below the output, it says 'Process finished with exit code 0'.

Очевидно, что число a больше, чем число b , и вроде бы логически здесь всё написано верно. Но если мы выполним этот код, то увидим, что результат будет обратным ожидаемому.

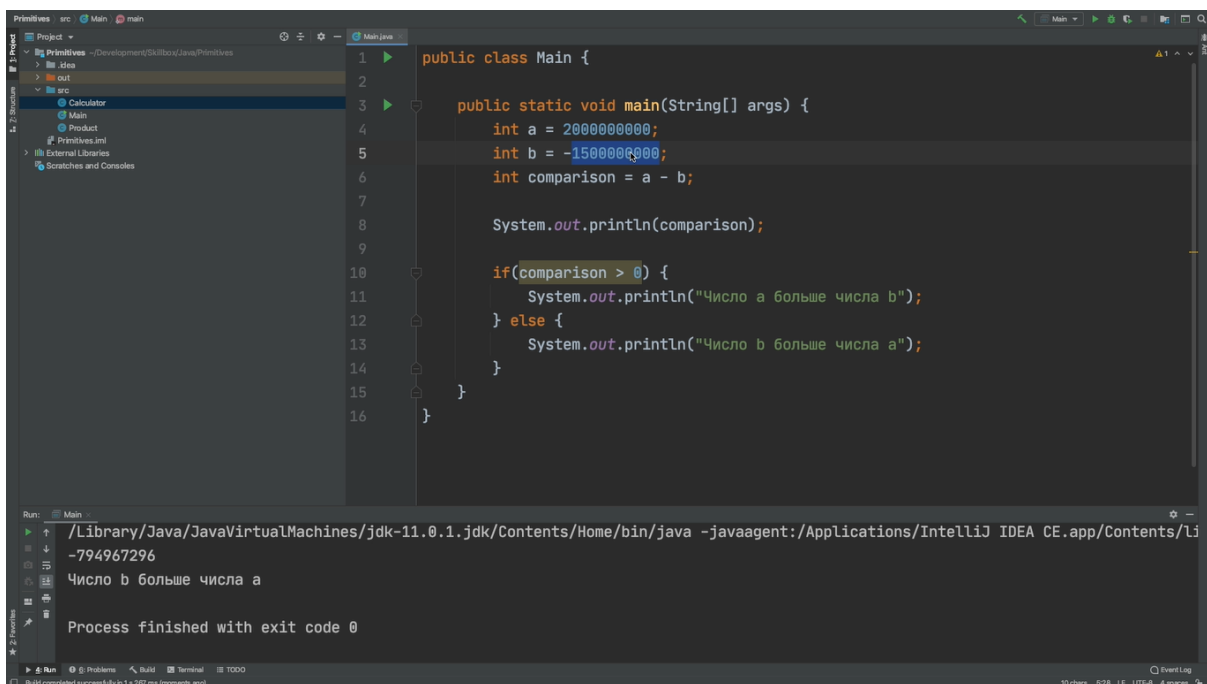


```
1 public class Main {
2
3     public static void main(String[] args) {
4         int a = 2000000000;
5         int b = -1500000000;
6         int comparison = a - b;
7         if (comparison > 0) {
8             System.out.println("Число a больше числа b");
9         } else {
10            System.out.println("Число b больше числа a");
11        }
12    }
13 }
```

Run: Main
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
Число b больше числа a
Process finished with exit code 0

Происходит это из-за того, что произошло переполнение.

Давайте посмотрим, чему равна переменная comparison: если из 2 млрд вычесть (-1,5 млрд), то должно получиться около 3 миллиардов, но поскольку значение числа `int` ограничено двумя с небольшим миллиардами, то мы получаем переполнение и такое странное отрицательное число.



```
1 public class Main {
2
3     public static void main(String[] args) {
4         int a = 2000000000;
5         int b = -1500000000;
6         int comparison = a - b;
7
8         System.out.println(comparison);
9
10        if (comparison > 0) {
11            System.out.println("Число a больше числа b");
12        } else {
13            System.out.println("Число b больше числа a");
14        }
15    }
16 }
```

Run: Main
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
-794967296
Число b больше числа a
Process finished with exit code 0

04:42–09:33

Числа с плавающей точкой

Теперь давайте поговорим о том, какими бывают и как устроены числа с плавающей точкой.

Их всего два — `float` и `double`. Единственное, чем они отличаются, — это своим размером в памяти: число `float` занимает в памяти 32 бита, то есть 4 байта, а число `double` — 64 бита, то есть 8 байт.

Название `float`, как мы уже обсуждали ранее, произошло от термина `floating point`, то есть «плавающая точка», а `double` в переводе с английского означает «двойной». Его так называли в связи с тем, что оно занимает в памяти в два раза больше места, чем `float`.

Давайте теперь посмотрим, какие значения могут принимать эти числа и как они устроены в памяти.

В памяти они хранятся в виде трёх частей — трёх компонентов. Первый компонент — это знак «плюс» или «минус», и он обозначается одним битом (как правило, 0 — это «плюс», а 1 — это «минус»). Второй компонент — это так называемая мантисса. Это основная часть числа, которая может включать в себя точку. Третий компонент — это порядок. По сути, это степень, в которую возводится некое основание — например, число 10, на которое потом умножается мантисса.

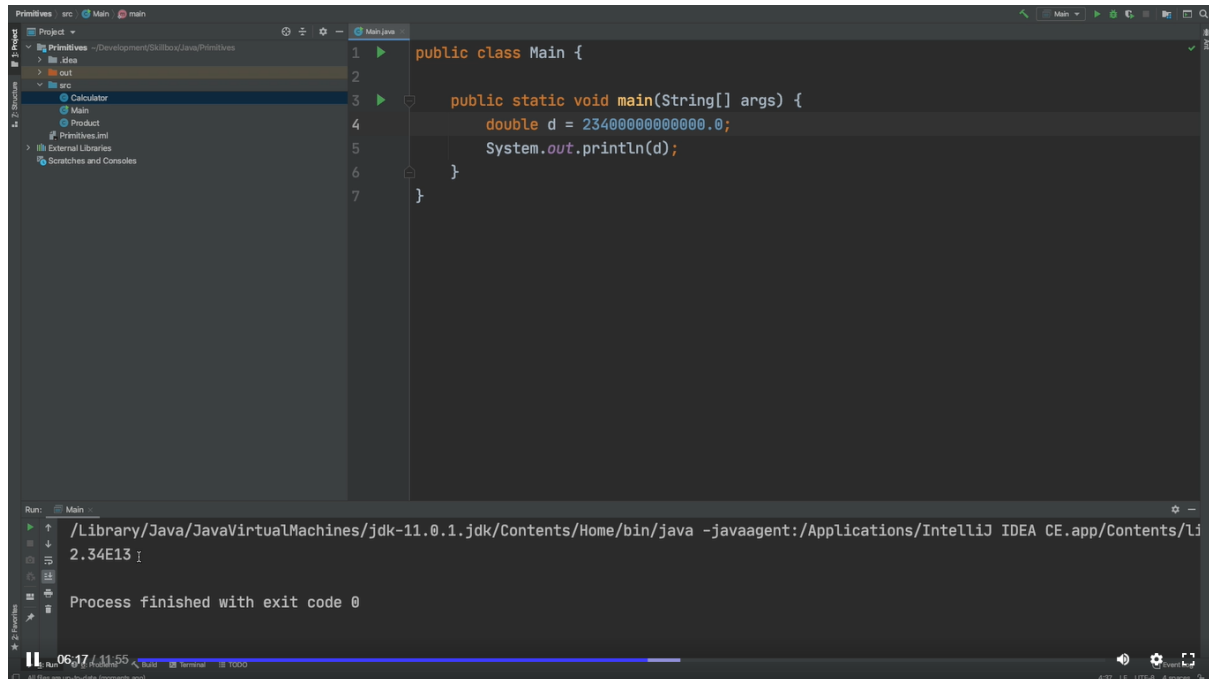
Числа с плавающей точкой. Структура



— 5.34234 × 10¹⁷

В памяти компоненты располагаются именно в таком порядке — сначала знак, потом порядок и только потом мантисса.

Если вы напишете любое число с плавающей точкой и выведете его в консоль, вы увидите все эти три компонента.

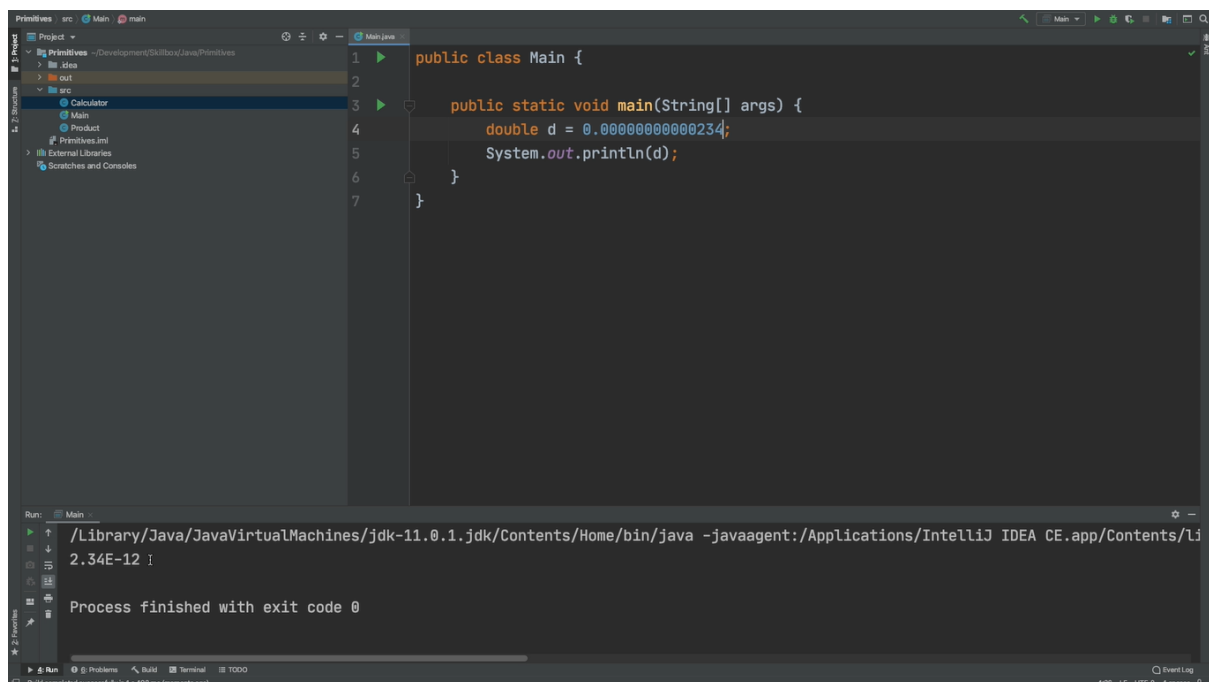


The screenshot shows the IntelliJ IDEA IDE with a project named 'Primitives'. The 'Main.java' file is open, containing the following code:

```
1 public class Main {  
2  
3     public static void main(String[] args) {  
4         double d = 2340000000000.0;  
5         System.out.println(d);  
6     }  
7 }
```

The Run console at the bottom shows the command: `/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li` and the output: `2.34E13`. Below the output, it says 'Process finished with exit code 0'.

Аналогичным образом можно вывести очень маленькое число таким образом, чтобы степень была отрицательной.



The screenshot shows the IntelliJ IDEA IDE with the same project and file. The code in 'Main.java' is now:

```
1 public class Main {  
2  
3     public static void main(String[] args) {  
4         double d = 0.00000000000234;  
5         System.out.println(d);  
6     }  
7 }
```

The Run console at the bottom shows the same command as before, but the output is `2.34E-12`. Below the output, it says 'Process finished with exit code 0'.

Все эти три компонента хранятся в памяти в двоичной системе счисления, то есть в виде последовательности нулей и единиц.

Чтобы понять, какие максимальные и минимальные значения могут принимать числа с плавающей точкой, нужно разобраться в том, сколько бит занимает каждый из этих компонентов.

Вы уже знаете, что числа типа `float` занимают в памяти 32 бита. Давайте посмотрим, как эти 32 бита распределяются по компонентам.

- 1 бит уходит на знак. Если это ноль — это положительное число, если 1, то число отрицательное.
- 8 бит уходят на порядок, из которых 7 бит — это, собственно, значение порядка и 1 бит — это знак порядка, поскольку степень может быть отрицательной.
- 23 бита выделено на мантиссу.

У числа `double`, которое в памяти занимает 64 бита, распределение немного другое:

- 1 бит — на знак;
- 11 бит — на порядок, из которых 10 битов, собственно, порядок и 1 бит — это знак;
- 52 бита — на мантиссу;

Числа с плавающей точкой. Структура



Давайте теперь посмотрим, какие минимальные и максимальные значения могут принимать эти числа. Начнём с числа float.

Для каждого числа с плавающей точкой есть два варианта максимальных и минимальных значений: это минимальное и максимальное положительное значение и минимальное и максимальное отрицательное значение.

Число «float»: минимум и максимум

Отрицательные числа	0	Положительные числа
		
Минимальное:		Минимальное:
$\approx -3,4 \times 10^{38}$		$\approx 1,4 \times 10^{-45}$
Максимальное:		Максимальное:
$\approx -1,4 \times 10^{-45}$		$\approx 3,4 \times 10^{38}$

Максимальное отрицательное и минимальное положительное находятся где-то очень близко к нулю.

Обратите внимание, что между максимальным отрицательным и минимальным положительным значениями также есть 0, которому число float может быть равно.

Теперь давайте посмотрим на минимальные и максимальные значения числа double — у них также четыре варианта.

Число «double»: минимум и максимум



Опять же между максимальным отрицательным и минимальным положительным числами есть значение 0, которому число double также может быть равно.

09:33–10:49

Способы записи чисел

Теперь давайте посмотрим на различные способы записи чисел. Есть несколько моментов, которые важно и полезно знать.

Первый момент, о котором хотелось бы рассказать, — это возможность при написании чисел помимо цифр писать ещё и **символы подчеркивания**. Это может быть удобно, например, для разделения чисел на разряды, чтобы повысить читабельность кода.

Второй важный момент: если вы задаёте число long, то после этого числа нужно писать букву l. Необходимость в явном виде обозначать число long буквой l может возникнуть тогда, когда записанное вами значение не уместается в число — например, если целое число слишком большое, поэтому вы должны обозначить его как long посредством добавления буквы l в конце числа.

Аналогичным образом можно поступить, если вы хотите в явном виде задать число типа `float`, — поставить в конце этого числа букву `f`. Чтобы задать число типа `double`, достаточно просто задать такое число, и если это число с нулём после точки, то можно поставить точку или точку и ноль.

Способы записи чисел

```
int count = 247_384;  
long big = 3235000000L;  
float value = 30.0f;  
double bigValue = 30.0;
```

10:49–11:29

Зачем столько видов?

Может возникнуть закономерный вопрос: зачем столько типов чисел? Почему бы не обойтись одним типом для целых чисел и одним типом для чисел с плавающей точкой?

Давайте ответим на эти вопросы последовательно.

Число `byte` используется довольно часто: его можно встретить в коде. А вот число `short` и число `float` можно встретить в коде достаточно редко. Зачем же они нужны?

Этими типами можно пользоваться для экономии памяти. Вы знаете, что число `short` занимает в два раза меньше места, чем число `int`, а число `float` — в два раза меньше, чем число `double`. Это может быть критичным при обработке очень больших объёмов данных или при разработке программного обеспечения для каких-нибудь устройств, на которых количество памяти ограничено.

11:29–11:55

Итоги

Итак, в этом видео вы познакомились с числами — как с целыми, так и с числами с плавающей точкой. Также вы узнали, сколько места они занимают в памяти и какие значения могут принимать. Кроме того, вы познакомились со способами записи этих чисел. В следующем видео мы познакомимся с типом `char`, то есть символами, которые, на самом деле, тоже могут быть представлены числами.